

JOURNAL 3

Frontend Implementation

Smart City Civic Complaint and Management System

Project	Smart City Civic Complaint and Management System
Development Stage	Frontend Prototype Implementation
Technology Used	React, Vite, React Router, JavaScript, CSS
State Management	React Context API
Student	Hittin

1. Objective

The work primarily focused on developing the Field Worker interface and the Administrator interface. The Field Worker interface was designed to support task acceptance, work progress tracking, evidence submission, and task completion. The Administrator interface was developed to provide an overview of complaints and their current processing status.

2. Development Environment and Technologies

The frontend components were developed using the following technologies:

- **React** – used for creating reusable and interactive frontend components.
- **Vite** – used as the frontend development environment and build tool.
- **JavaScript** – used for implementing component logic and user interactions.
- **React Router** – used for navigation between dashboard and task-related pages.
- **CSS** – used for designing dashboards, cards, buttons, progress indicators, forms, and status displays.
- **React Context API** – used for accessing and updating shared complaint information between frontend components.

3. Field Worker Dashboard

The Field Worker Dashboard was developed to provide field workers with a centralized interface for viewing their assigned civic complaint tasks.

The dashboard provides information about assigned work and the current progress of tasks.

The main sections include:

- Assigned Tasks
- In Progress Tasks
- Completed Tasks
- Complaint task information
- Task status
- Location information
- Task details navigation

Each task provides a mechanism to open its detailed task interface.

This allows the field worker to move from the dashboard to the individual task workflow.

4. Worker Task Details

A separate *Worker Task Details* page was developed for handling the complete lifecycle of an assigned field task.

The task details interface displays important information about the civic complaint and provides controls for updating the work status.

The implemented workflow is:

Assigned → Accepted → In Progress → Completed

Each stage represents a different point in the field-worker workflow.

5. Accept Task

An *Accept Task* operation was implemented for field workers.

When a worker accepts an assigned task, the task status changes from *Assigned* to *Accepted*.

This action represents that the field worker has received and accepted the responsibility for performing the assigned work.

The interface dynamically reflects the updated status after the action.

6. Start Work

After accepting a task, the field worker can start the assigned work.

The *Start Work* operation changes the task status to *In Progress*.

Initial progress is also set when work begins.

This provides a clear distinction between a task that has only been accepted and a task on which actual field work has started.

7. Work Progress Tracking

A progress tracking section was implemented to allow the field worker to update the percentage of work completed.

A progress slider was provided in the task interface.

The progress value represents the current completion level of the field work.

For example:

- 0% – Work has not started.
- 10% – Work has recently started.
- 50% – Approximately half of the work is completed.
- 100% – Work is completely finished.

The interface also contains a visual progress bar so that the current completion percentage can be understood easily.

8. Work Notes

A work-notes section was implemented to allow the field worker to provide additional information about the performed work.

The worker can enter notes describing activities carried out during the resolution process.

Examples of information that can be recorded include:

- Work performed.
- Issue identified at the location.

- Materials or actions used for resolution.
- Additional observations.

These notes provide supporting information before the task is marked as completed.

9. Evidence Upload

An evidence-upload interface was implemented for the field worker.

The worker can select an evidence file after carrying out the assigned work.

The interface displays the selected file so that the worker can verify that evidence has been attached.

Evidence is intended to support the completion of a civic complaint by providing a record of the work performed.

At the current frontend stage, the evidence file is handled through the frontend interface and is not permanently stored in a backend database.

10. Task Completion Validation

Validation was implemented before allowing a field worker to complete a task.

The task cannot be marked as completed unless the required work information has been provided.

The completion condition requires:

- Work progress to reach 100%.
- Work notes to be entered.

This prevents a task from being marked as completed prematurely.

The validation provides a basic frontend-level control for maintaining the correct task workflow.

11. Complete Task

After the work progress reaches 100% and the required work notes have been provided, the field worker can select the *Complete Task* operation.

The task status is then changed to:

Completed

The completed-task interface provides a clear indication that the field work has reached its final stage.

This status can subsequently be used by the complaint workflow to indicate that the resolution is ready for further processing.

12. Worker Interface Styling

CSS styling was implemented for the Field Worker interface to improve readability and usability.

The styling covers:

- Worker dashboard layout.
- Task cards.
- Status indicators.
- Progress slider.
- Progress bar.
- Work-notes section.
- Evidence-upload section.
- File selection display.
- Task action buttons.
- Completed-task interface.

The interface was tested in the browser to verify that the different task states and controls were displayed correctly.

13. Administrator Dashboard

An Administrator Dashboard was developed to provide an overall view of the complaint management system.

The administrator can view complaint information and monitor the current state of complaints through a centralized dashboard.

The dashboard displays complaint-related statistics and a complaint list.

14. Administrator Statistics

Dynamic complaint statistics were implemented in the Administrator Dashboard.

The dashboard calculates the number of complaints belonging to different status categories.

The implemented statistics include:

- Total Complaints
- Pending Complaints
- In Progress Complaints
- Completed Complaints
- Resolved Complaints
- Reopened Complaints

These values are calculated from the complaint collection available through the shared frontend state.

Therefore, the displayed values change when complaint information changes.

15. Administrator Complaint Management View

A complaint listing was implemented in the Administrator Dashboard.

For each complaint, the administrator can view important information such as:

- Complaint ID
- Complaint title
- Complaint category
- Location
- Submission date
- Current status
- Assigned worker

This provides the administrator with a consolidated view of complaints being handled by the system.

16. Shared State Integration

The Administrator Dashboard was connected with the shared complaint state implemented using React Context API.

The dashboard obtains the complaint collection using the `useComplaints()` hook.

The shared state contains information such as:

- Complaint status
- Assigned worker
- Work progress
- Work notes
- Evidence information
- Citizen verification status

This allows the administrator interface to display the current frontend state of complaints rather than maintaining a completely separate complaint dataset.

17. Frontend Navigation

The Field Worker and Administrator interfaces were connected to the application routing structure.

The major routes associated with these interfaces include:

- Worker Dashboard
- Worker Task Details
- Administrator Dashboard

React Router was used to navigate between these pages.

The worker can open an individual task from the dashboard, while the administrator can access the centralized complaint overview.

18. Testing and Debugging

The implemented interfaces were tested through the React development server.

The following operations were verified:

- Opening the Field Worker Dashboard.
- Opening individual worker tasks.
- Accepting an assigned task.
- Starting a task.
- Updating work progress.
- Entering work notes.
- Selecting evidence files.
- Completing a task after satisfying validation conditions.
- Opening the Administrator Dashboard.
- Displaying complaint statistics.
- Displaying complaint information in the administrator view.
- Navigating between the implemented frontend routes.

CSS and component-level issues encountered during development were corrected and the interfaces were verified in the browser.

19. Current Frontend Result

The Field Worker and Administrator frontend modules were successfully implemented as part of the individual contribution.

The Field Worker module supports the complete prototype task progression:

Assigned → Accepted → In Progress → Completed

The Administrator module provides a centralized overview of complaints, including dynamic complaint statistics and complaint status information.

These modules form the staff-side portion of the frontend prototype.

20. Current Limitations

The current frontend implementation has the following limitations:

- Complaint information is maintained in frontend state and is not permanently stored after a page refresh.
- Worker authentication is currently represented through the frontend prototype.
- Evidence files are selected through the frontend but are not yet permanently stored in a backend server.
- Administrator operations currently focus on viewing and monitoring complaint information.
- Backend API and PostgreSQL database integration are not part of this frontend development stage.

21. Learning Outcomes

The development of the Field Worker and Administrator modules provided practical understanding of:

- React component development.
- React Router based navigation.
- Frontend state management.
- Dynamic rendering based on status values.
- Interactive progress controls.
- Form and input handling.
- Basic frontend validation.
- File selection and evidence handling.
- Dashboard development.
- Dynamic calculation of dashboard statistics.
- CSS-based interface design.
- Frontend testing and debugging.